МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №4
по дисциплине
«Программирование»

Выполнил студент гр. ИВТб-1304-06-00 _____ /Золотухина Ю.А./

Проверил старыший преподаватель _____ /Крутиков А.К./

Киров
2025

Задание

Разработать консольное приложение для демонстрации особенностей организации элементарных структур данных. Для организации взаимодействия с пользователем использовать case-меню.

Цель работы

Изучение механизмов эффективного управления динамической памятью, создание элементарных структур данных.

Требования

1) интерфейс взаимодействия с пользователем организован как меню. Навигация в меню реализуется через клавиши (в виде стрелок)

2) применяются ТОЛЬКО структуры данных динамической памяти

3) работа пунктов меню в коде реализована через применение подпрограмм

4) предусмотреть отдельные пункты меню для добавления и удаления элементов динамической структуры

5) предлагаемые языки программирования: Pascal, C.

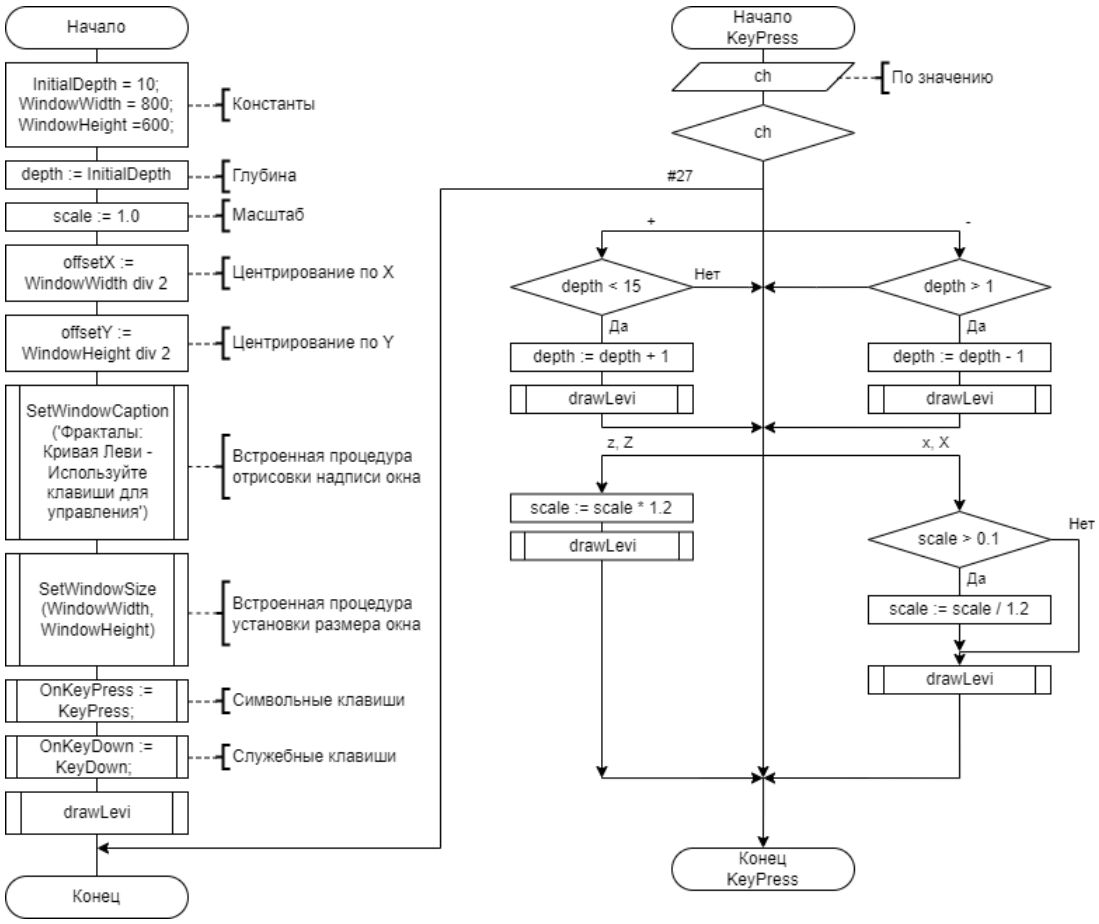

Рисунок 1 – 1 страница

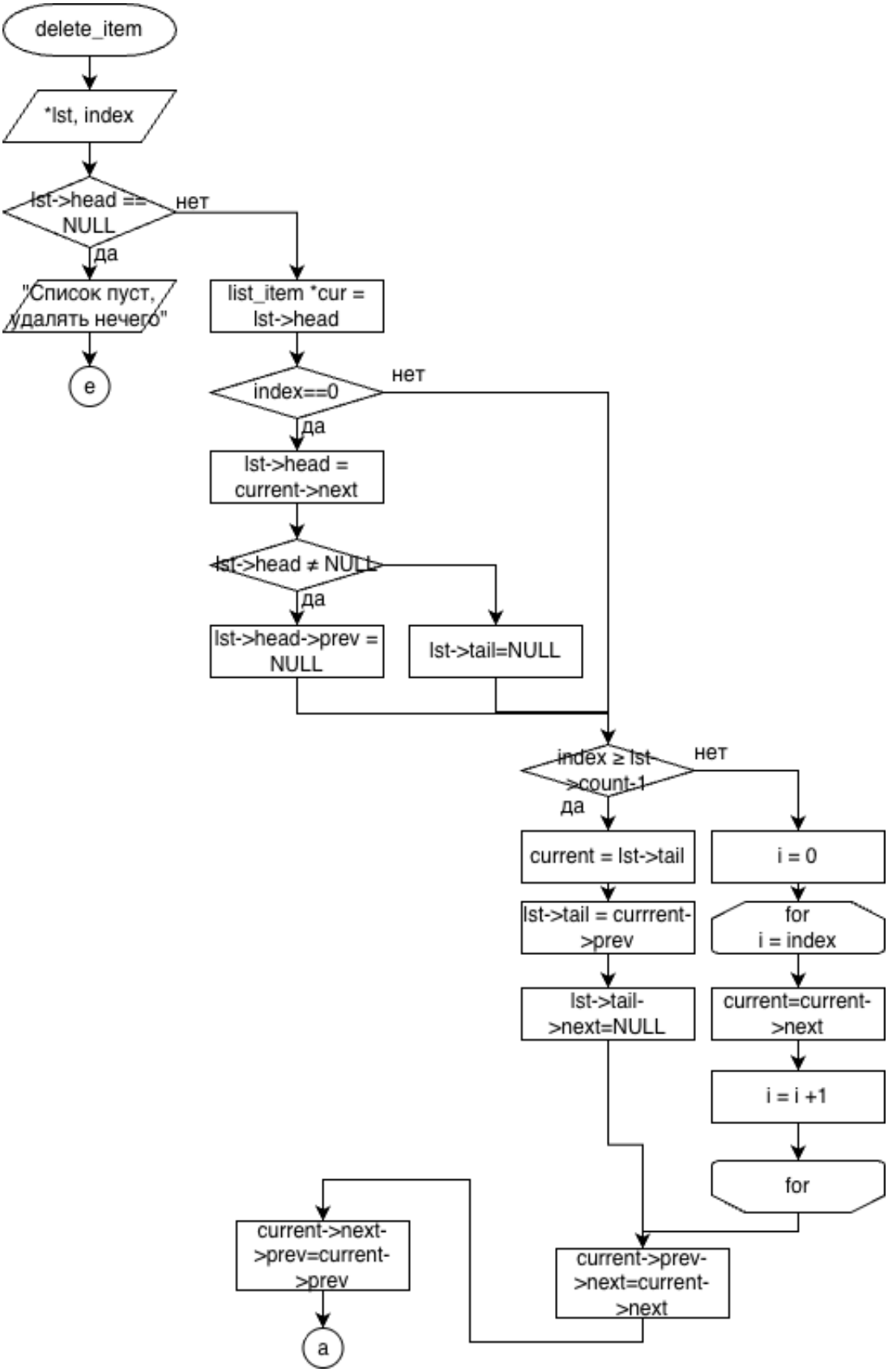

Рисунок 2 – 2 страница

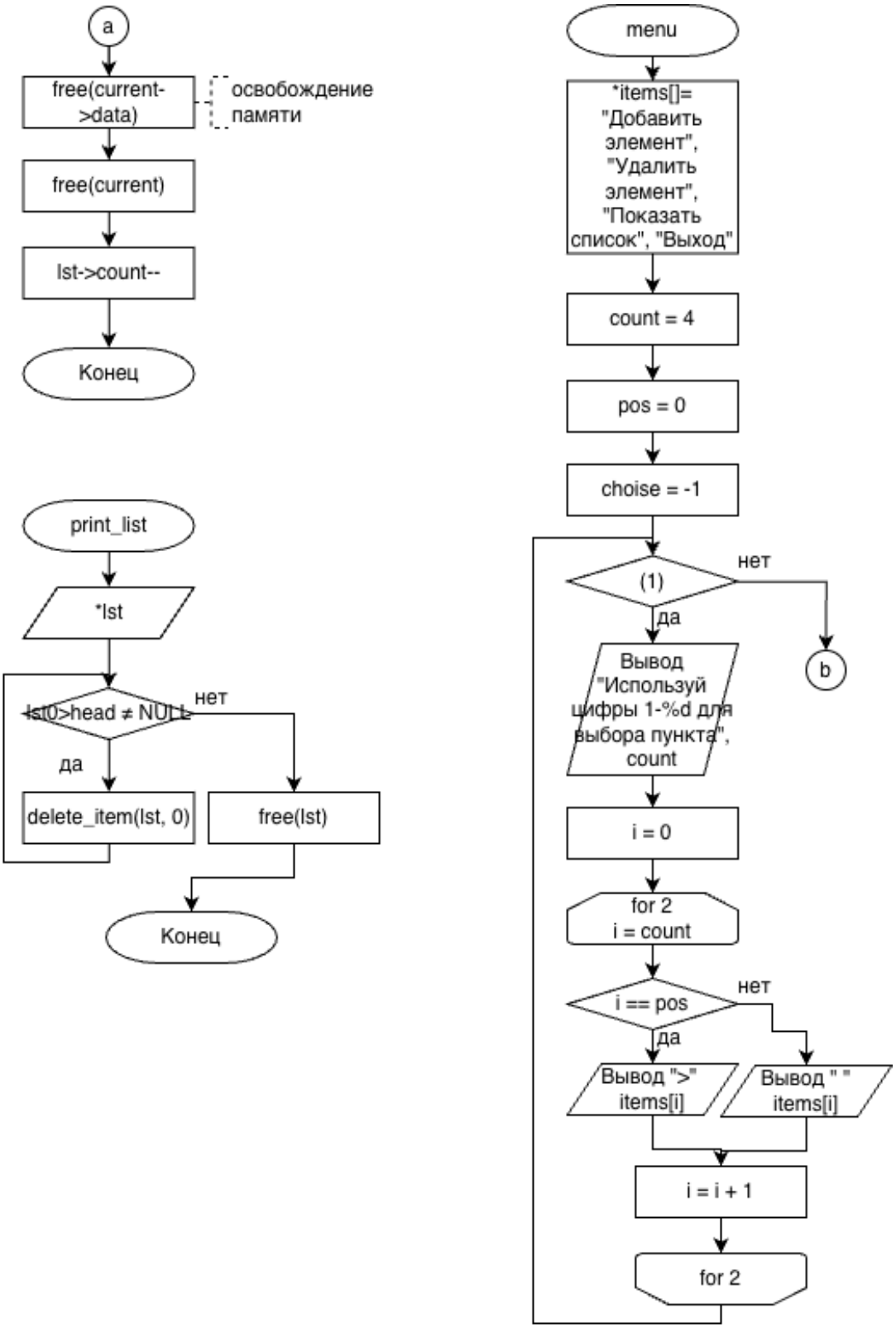

Рисунок 3 – 3 страница

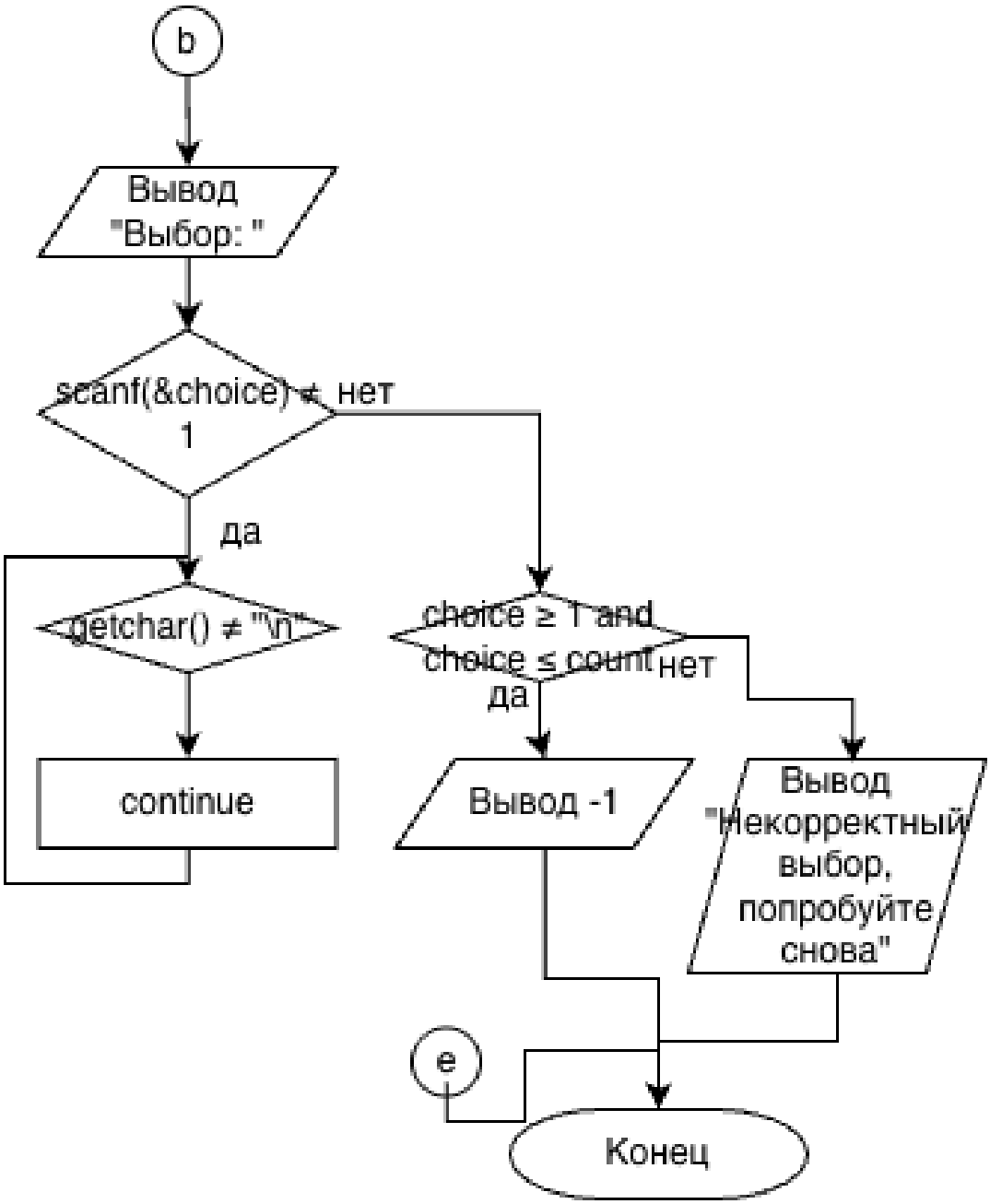

Рисунок 4 – 4 страница

Решение

```

#include <stdio.h>
#include <stdlib.h> //функции для работы с динамической памятью (malloc, free)
#include <string.h> //функции для работы с динамической памятью (malloc, free)

// структура узла списка
typedef struct list_item {
    char *data;
    struct list_item *next;
    struct list_item *prev;
} list_item;

// структура самого списка
typedef struct list {
    int count;
    list_item *head;
    list_item *tail;
} list;

// функция создания пустого списка
list* create_list() {
    list *lst = (list*)malloc(sizeof(list));
    lst->count = 0;
    lst->head = NULL;
    lst->tail = NULL;
    return lst;
}

// функция добавления элемента в список
void insert_list(list *lst, int index, char *data) {
    list_item *new_item = (list_item*)malloc(sizeof(list_item));
    new_item->data = (char*)malloc(strlen(data) + 1);
    strcpy(new_item->data, data);
```

```
    new_item->next = NULL;
    new_item->prev = NULL;

    if (lst->head == NULL) { // список пустой
        lst->head = new_item;
        lst->tail = new_item;
    } else if (index == 0) { // вставка в начало
        new_item->next = lst->head;
        lst->head->prev = new_item;
        lst->head = new_item;
    } else if (index >= lst->count) { // вставка в конец
        new_item->prev = lst->tail;
        lst->tail->next = new_item;
        lst->tail = new_item;
    } else { // вставка в середину
        list_item *current = lst->head;
        for (int i = 0; i < index; i++) {
            current = current->next;
        }
        new_item->next = current;
        new_item->prev = current->prev;
        if (current->prev != NULL) {
            current->prev->next = new_item;
        }
        current->prev = new_item;
    }

    lst->count++;
    printf("Добавлен элемент: %s\n", data);
}

// функция удаления элемента по индексу
void delete_item(list *lst, int index) {
    if (lst->head == NULL) {
        printf("Список пуст, удалять нечего.\n");
        return;
    }
```

```c

    list_item *current = lst->head;
    if (index == 0) { // удаление головы
        lst->head = current->next;
        if (lst->head != NULL) {
            lst->head->prev = NULL;
        } else {
            lst->tail = NULL; // список стал пустым
        }
    } else if (index >= lst->count - 1) { // удаление хвоста
        current = lst->tail;
        lst->tail = current->prev;
        lst->tail->next = NULL;
    } else { // удаление из середины
        for (int i = 0; i < index; i++) {
            current = current->next;
        }
        current->prev->next = current->next;
        current->next->prev = current->prev;
    }

    free(current->data);
    free(current);
    lst->count--;
}

// функция вывода списка
void print_list(list *lst) {
    printf("Список (%d элементов):\n", lst->count);
    list_item *current = lst->head;
    int i = 0;
    while (current != NULL) {
        printf("%d: %s\n", i, current->data);
        current = current->next;
        i++;
    }
}
```

```c

// функция очистки списка
void free_list(list *lst) {
    while (lst->head != NULL) {
        delete_item(lst, 0);
    }
    free(lst);
}

// меню для работы с пользователем
int menu() {
    const char *items[] = {"Добавить элемент", "Удалить элемент", "Показать
    ↪ список", "Выход"};
    int count = 4;
    int pos = 0;
    int choice = -1;

    while (1) {
        printf("\nИспользуйте цифры 1-%d для выбора пункта:\n", count);
        for (int i = 0; i < count; i++) {
            if (i == pos) {
                printf("> %s\n", items[i]);
            } else {
                printf("  %s\n", items[i]);
            }
        }

        printf("Выбор: ");
        if (scanf("%d", &choice) != 1) {
            while (getchar() != '\n'); // очищаем буфер
            continue;
        }

        if (choice >= 1 && choice <= count) {
            return choice - 1; // возвращаем индекс пункта
        } else {
            printf("Некорректный выбор, попробуйте снова.\n");
```

```
        }
    }
}

int main() {
    list *lst = create_list();
    int running = 1;

    while (running) {
        int choice = menu();
        if (choice == 0) { // добавить элемент
            char buffer[100];
            printf("Введите данные: ");
            scanf("%s", buffer);
            insert_list(lst, lst->count, buffer);
        } else if (choice == 1) { // удалить элемент
            int index;
            printf("Введите индекс для удаления: ");
            scanf("%d", &index);
            delete_item(lst, index);
        } else if (choice == 2) { // показать список
            print_list(lst);
        } else if (choice == 3) { // выход
            running = 0;
        }
    }

    free_list(lst);
    return 0;
}


```

Ссылка на GitHub

https://github.com/yulzolo/lab4.git

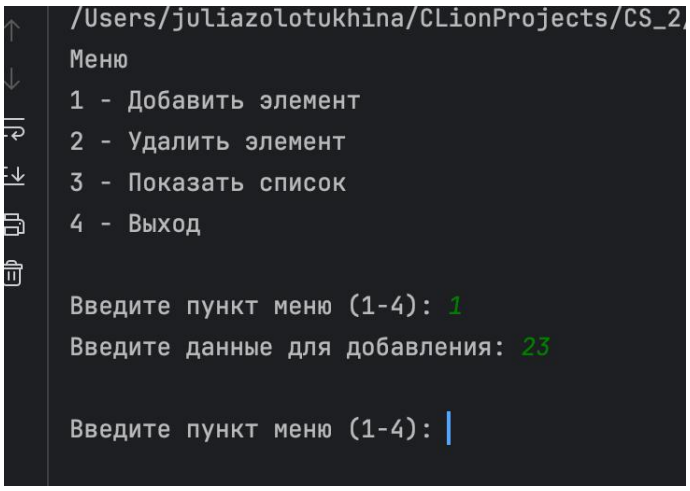

Рисунок 5 – Терминал

Вывод

В ходе выполнения лабораторной работы №3 были получены знания работы с оператором ветвления switch-case, а также навыки разработки консольных программ на языке программирования C. В рамках работы была реализована программа для вычисления площади фигуры, ограниченной кривой. Вычисление площади фигуры осуществлялось путём нахождения определённого интеграла функции на заданном промежутке. Для численного интегрирования был применён метод трапеций, основанный на разбиении интервала интегрирования на равные отрезки и аппроксимации площади под кривой суммой площадей трапеций. Пределы интегрирования задавались пользователем в процессе выполнения программы. Для управления работой программы и выбора выполняемых операций было реализовано текстовое меню, функционирующее на основе оператора switch-case. Данный подход обеспечил наглядное и удобное взаимодействие пользователя с программой. В программе были реализованы отдельные функции для вычисления значения подынтегральной функции и её первообразной, что позволило структурировать код и повысить его читаемость. Для вычисления определённого интеграла использовалась процедура, выполняющая суммирование значений функции в узлах разбиения с учётом шага интегрирования. В программе были реализованы отдельные функции для вычисления значения подынтегральной функции и её первообразной, что позволило структурировать код и повысить

его читаемость. Для вычисления определённого интеграла использовалась процедура, выполняющая суммирование значений функции в узлах разбиения с учётом шага интегрирования. Оценка погрешности численного метода выполнялась путём сравнения результата, полученного методом трапеций, с точным значением интеграла, вычисленным на основе первообразной функции. Абсолютная погрешность определялась как модуль разности между точным и приближённым значениями интеграла.